Part C — Sequence models & attention

ml-ladder writeup · part C

2026-09-18

Contents

C1 — Char-level RNN and LSTM **2**
The math 2

C2 — Seq2seq with and without attention **5**
The math 5

C3 — A transformer from scratch **8**
The math 8

C4 — Fine-tuning a small ViT **11**
The math 11

C5 — What changed since C3, and what it cost **14**
The math 14

What part C settled **17**

Status: complete (C1–C5) · **Constraint:** subsets and short budgets; C1/C3 share every setting so they can be compared
Numbers: seeds 0–2, reference seed 0 · raw values in `data/c-metrics.json`
Reproduce: `ladder run C1 ... C4` (needs `uv sync --extra torch`), or `uv run python tools/writeup_figures.py C`
Hardware: this part's sweep ran on a Colab T4 — C4 is ~25 min/seed on four CPU cores against ~35 s on the GPU

Part B's models all see their whole input at once. Part C's have to decide what to *carry*: a recurrent cell squeezes the past into a fixed state, attention keeps everything and learns where to look, and a ViT applies the second answer to images. Four rungs, one question —

where does the information have to travel, and what does the architecture charge for the trip? A vanilla RNN charges a multiplicative factor per step, which is why its gradient dies. An LSTM buys an additive path, the same trick as B4's residual connection, one year earlier and in time rather than depth. Attention refuses to compress at all and pays quadratic cost instead. Every result below is a price for distance.

Notation: T is sequence length, h_t the hidden state at step t, $Q, K, V \in \mathbb{R}^{T\times d}$ the attention projections, and d_k the per-head width.

C1 — Char-level RNN and LSTM

Question. The LSTM is supposed to remember further than a vanilla RNN. How much further, and does the extra memory show up as a better language model?

The math

Why the vanilla gradient dies. With $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$, backpropagation through time multiplies one Jacobian per step:

$$\frac{\partial h_T}{\partial h_t} = \prod_{i=t+1}^{T} \frac{\partial h_i}{\partial h_{i-1}} = \prod_{i=t+1}^{T} \operatorname{diag}(1 - h_i^2)\, W_h^\top .$$

Taking norms, $\|\partial h_T/\partial h_t\| \le (\gamma\|W_h\|)^{T-t}$ where $\gamma = \max_i \|\operatorname{diag}(1-h_i^2)\| \le 1$ for tanh. The bound is **geometric in the distance**: if $\gamma\|W_h\| < 1$ the gradient vanishes exponentially, and if it exceeds 1 the gradient explodes instead. There is no setting of W_h that keeps a long-range signal alive without risking the other failure — which is the whole problem, and why the rungs clip gradients.

What the LSTM changes. The cell state's recurrence is additive:

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \quad \Longrightarrow \quad \frac{\partial c_t}{\partial c_{t-1}} = \operatorname{diag}(f_t).$$

The product over steps becomes $\prod_i \operatorname{diag}(f_i)$ — no weight matrix, no saturating derivative, just the forget gates. A gate near 1 passes gradient through essentially unchanged, and the network *learns* which of them to open. That is why the forget-gate bias is initialized to 1: start by remembering.

Measuring it. The rung differentiates a scalar that depends only on the final state and reports $\|\partial L_T/\partial x_t\|$ against distance, then defines the **effective horizon** as the largest distance at which that norm is still above 10^{-6} of its value at the final position.

Results (tiny Shakespeare, 250k characters, 64-character context, 500 steps; medians over seeds 0–2):

model	parameters	bits/char	horizon	gradient at 63 steps back
vanilla RNN	98k	2.366	27	1.4e-11
vanilla RNN (width-matched)	343k	**2.263**	27	6.1e-11
LSTM	320k	2.334	**63**	9.4e-03

The horizon gap is the cleanest measurement in the part: 63 against 27 on every seed, and about 10^9 **times more gradient** at 63 steps back.

What surprised me. *The LSTM loses the language-modelling comparison.* At a matched parameter budget the width-matched vanilla RNN is 0.07 bits/char better, consistently across seeds (2.261–2.274 against 2.319–2.336, non-overlapping). The memory is real and does not pay, because next-character prediction in Shakespeare at a 64-character context is dominated by short-range structure — spelling, morphology, the line break after a speaker's name. Nothing in that needs to reach 63 steps back, and the LSTM spends four gates' worth of parameters per unit of state to buy reach the task does not use.

So the rung grew a second experiment: a **recall probe** where reach is the only thing that matters. The label is the sequence's first symbol, every later position is filler, and chance is 1/8:

sequence length	vanilla RNN	LSTM
5	1.000	1.000
15	0.127 (chance)	**1.000**
30	0.127	0.115 (chance)
60	0.127	0.115

There it is, and also its limit: the LSTM solves 15-step recall where the RNN has already collapsed to chance — about **3× the reach** — but at 30 steps it fails too, at this budget (120 steps, hidden 64). The horizon measurement says gradient *arrives* at 63 steps; the recall probe says arriving is not the same as learning to use it. I would rather report that than quietly pick the length where the LSTM still wins.

Open threads. GRUs, to see how much of the LSTM's benefit survives with one gate fewer. Truncated BPTT over a longer context, which is how these models were actually trained. And the recall probe with a proper budget sweep, to find where the LSTM's 63-step gradient becomes 63-step learning.

Reading: Long Short-Term Memory (Hochreiter & Schmidhuber); Pascanu et al., *On the difficulty of training recurrent neural networks*; Karpathy, *The Unreasonable Effectiveness of RNNs*.

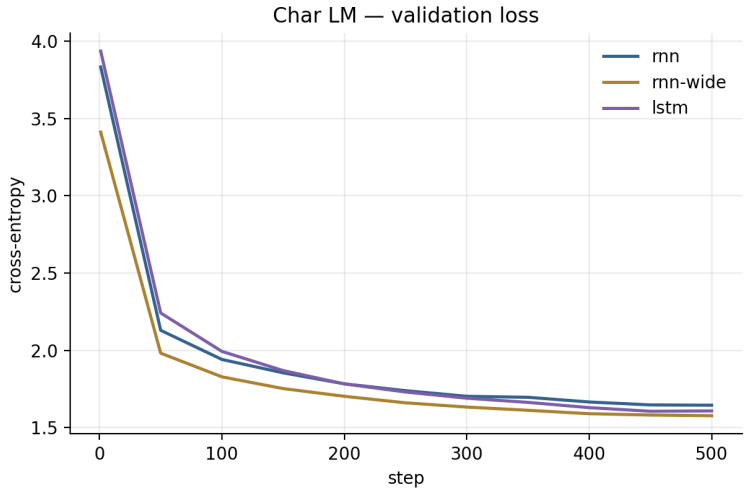


Figure 1: Validation loss

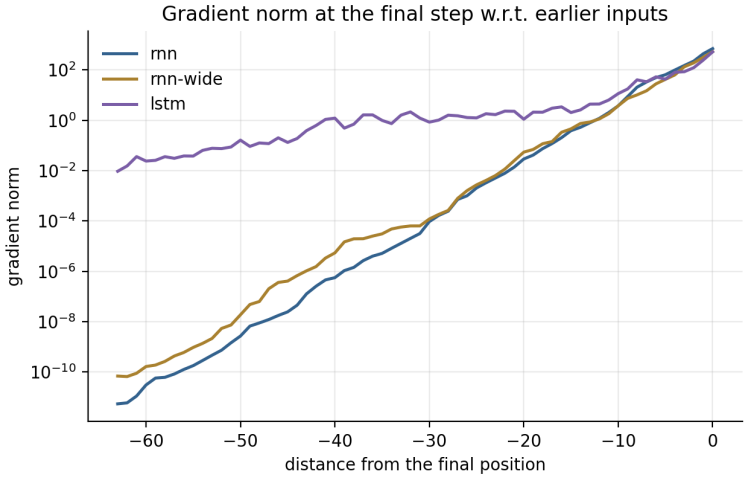


Figure 2: Gradient reach

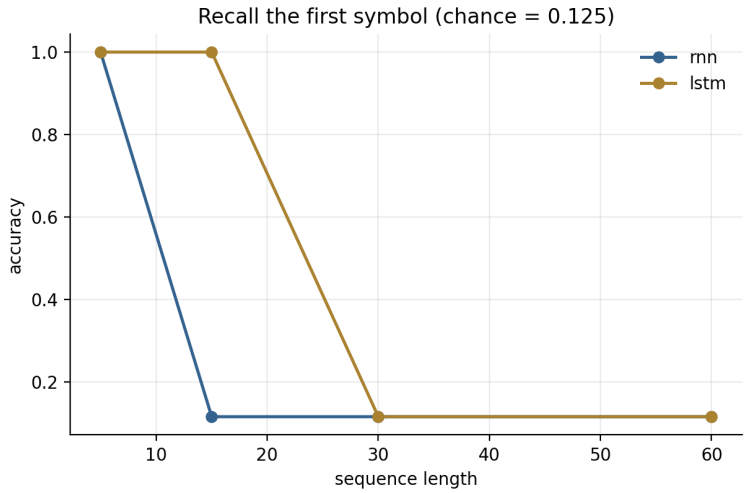


Figure 3: Recall accuracy

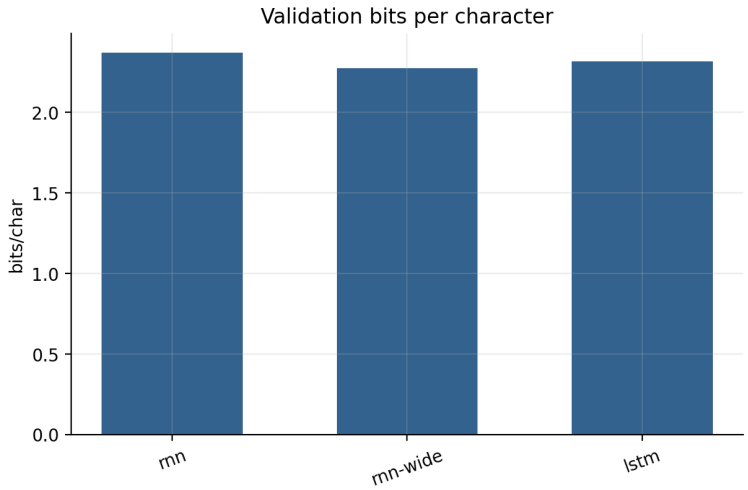


Figure 4: Bits per character

C2 — Seq2seq with and without attention

Question. How much does forcing a whole source sentence through one fixed-size vector actually cost?

The math

The bottleneck. A classic encoder-decoder compresses the source to a single vector $c = h_{T_s}$ and generates from it:

$$p(y_1, \dots, y_{T_t} \mid x) = \prod_j p\big(y_j \mid y_{<j},\, c\big)\,.$$

Everything the decoder can ever know about the source must fit in c, whose size is fixed while T_s grows. With d-dimensional states the channel carries $O(d)$ numbers regardless of input length — so the cost of the bottleneck should grow with the amount of *information* in the source, not its padded length. (The first version of this rung missed exactly that, swept padding, and found nothing.)

Attention removes the constraint. Keep every encoder state and build a fresh context per output step:

$$e_{ji} = \text{score}(s_j, h_i), \qquad \alpha_{ji} = \frac{\exp e_{ji}}{\sum_{i'} \exp e_{ji'}}, \qquad c_j = \sum_i \alpha_{ji} h_i.$$

This rung uses Luong's dot product, $\text{score}(s, h) = s^\top h$ (Bahdanau's additive $v^\top \tanh(W[s; h])$ is the other standard choice). The context is now a *convex combination of all source states*, so no fixed budget limits what reaches the decoder; the price is $O(T_s T_t)$ scores instead of $O(T_t)$, and it is the same price the transformer pays in C3.

Padding must be masked **before** the softmax, by setting those logits to $-\infty$: masking afterwards would leave the remaining weights un-normalized.

Why this task. Every output character of an ISO date is a copy of some input span, so the correct alignment is known in advance and the attention map can be *scored* — the fraction of output

characters whose attention peak sits on a matching input character — rather than admired.

Results (date normalization, filler words around the date, medians over seeds 0–2):

filler	source length	bottleneck exact	attention exact	bottleneck char-level
none	18	0.997	1.000	0.999
10 words	101	0.326	1.000	0.899
24 words	193	**0.005**	**1.000**	0.710

The attention model is perfect at every length; the bottleneck model falls off a cliff. Alignment score 0.729 median — high, and *not* 1.0 by construction, since the month digits must come from a month *name* and count as misses.

What surprised me. The gap between the two accuracy measures. At 193 characters the bottleneck model still gets **71% of characters** right while getting **0.5% of dates** right. It has learned the *shape* of the answer — four digits, dash, two digits, dash, two digits, roughly the right ranges — and lost the content. That is the most honest picture of what a too-small channel does: not noise, but confident, well-formed, wrong output. Character accuracy would have made this look like a mild degradation; exact match shows it is total.

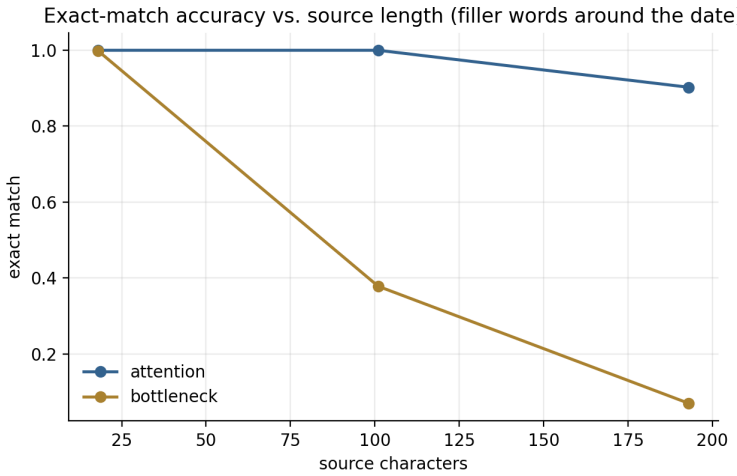


Figure 5: Accuracy by source length

Open threads. Bahdanau's additive score, which has an extra parameter matrix and is claimed to matter for longer sources. Coverage penalties, the standard fix for attention that revisits one position and drops another. And beam search instead of greedy decoding, which should matter most exactly where the bottleneck model is uncertain.

Reading: Neural Machine Translation by Jointly Learning to Align and Translate (Bahdanau et al.); Effective Approaches to Attention-based NMT (Luong et al.); Sequence to Sequence Learning (Sutskever et al.).

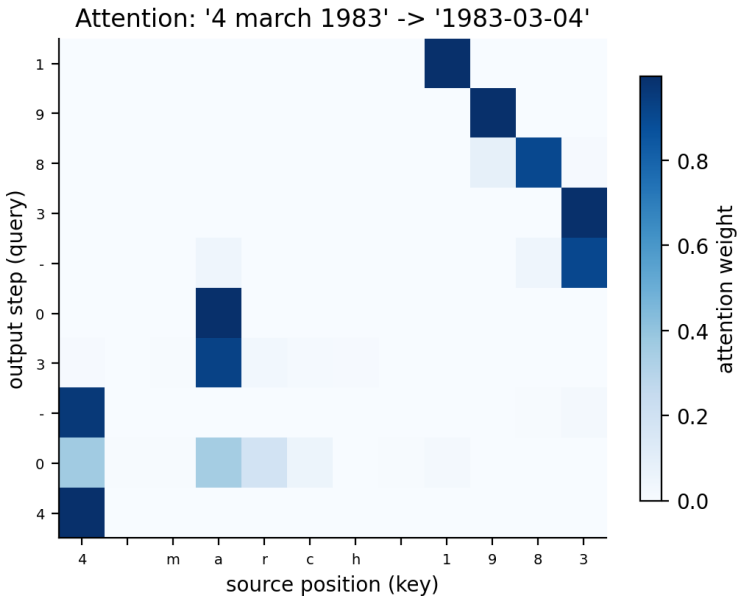


Figure 6: Attention map

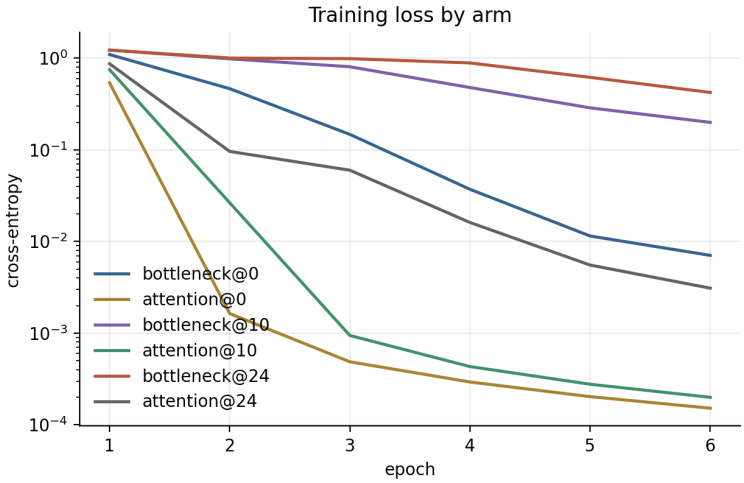


Figure 7: Training loss

C3 — A transformer from scratch

Question. Does the implementation actually respect causality — and how much of a transformer's ability is the positional encoding?

The math

Scaled dot-product attention, and why the scale is there:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V.$$

If the entries of q and k are independent with zero mean and unit variance, then $q^\top k = \sum_{i=1}^{d_k} q_i k_i$ has variance d_k. Without the $1/\sqrt{d_k}$, logits grow like $\sqrt{d_k}$, the softmax saturates toward a one-hot distribution, and its Jacobian $\text{diag}(p) - pp^\top$ goes to zero — attention stops learning precisely when the model is made wider. Dividing by $\sqrt{d_k}$ holds the logit variance at 1 regardless of head width.

The causal mask is $M_{ij} = -\infty$ for $j > i$ and 0 otherwise, applied *before* the softmax so the forbidden positions get exactly zero weight rather than a small one. This is trivially easy to get subtly wrong — off by one, or masked after the softmax — and a model that leaks a single future position still trains beautifully while being worthless. The rung therefore checks it: perturb the last token, and no earlier logit may move.

Positional encoding, and what happens without it. Self-attention is *permutation-equivariant*: for a permutation matrix P,

$$\text{Attention}(PQ, PK, PV) = P\,\text{Attention}(Q, K, V),$$

so a transformer with no positional information cannot distinguish "abc" from "cba" — with a causal mask it degenerates to a function of the *multiset* of preceding characters. The sinusoidal encoding

$$PE_{(t,2i)} = \sin\left(\frac{t}{10000^{2i/d}}\right), \qquad PE_{(t,2i+1)} = \cos\left(\frac{t}{10000^{2i/d}}\right)$$

fixes this with a useful extra property: for a fixed offset k, the pair $(\sin\omega t, \cos\omega t)$ maps to $(\sin\omega(t+k), \cos\omega(t+k))$ by a rotation that depends on k but **not** on t, so "k steps back" is a linear operation the attention can express uniformly across the sequence.

Pre-LN blocks. The rung normalizes inside the residual branch, $x \mapsto x + f(\text{LN}(x))$, rather than the 2017 paper's $x \mapsto \text{LN}(x + f(x))$. Post-LN puts a normalization on the skip path itself and needs learning-rate warmup to train at all; pre-LN keeps the identity path clean — B4's argument again.

Results (same corpus, split, eval windows, steps, batch and context as C1; medians over seeds 0–2):

variant	parameters	bits/char
learned positions	1.82M	**2.309**

variant	parameters	bits/char
sinusoidal positions	1.80M	2.349
no positional encoding	1.80M	**2.641**
learned positions, at C1's budget	242k	2.508

Removing position costs **0.304 bits/char** (range 0.265–0.348) — the largest ablation in the part, and exactly what the equivariance argument predicts. Learned beats sinusoidal by 0.040 at this scale. The causal check returns **exactly 0.0** on every seed, for both the logit perturbation and the attention mass above the diagonal.

What surprised me — and the result I cannot explain away. Line up C1 and C3, which share every training setting:

model	parameters	bits/char
vanilla RNN (wide)	343k	**2.263**
transformer, learned PE	1.82M	2.309
LSTM	320k	2.334
transformer at C1's budget	242k	2.508

The wide vanilla RNN beats every transformer arm, using 5× fewer parameters than the best of them, and the parameter-matched transformer is the worst model in the comparison. That is not the story anyone expects in 2026, and it is not a claim that transformers are worse — it is a claim about this regime: 500 steps, 64 characters of context, 250k characters of training text, and *no warmup* for the transformer, which B5 showed is the one thing Adam-family optimizers are picky about at the start of training. Transformers are built to exploit long context and large data, and this rung deliberately gives them neither. The honest summary is that I measured a regime where the inductive bias of recurrence is worth more than the flexibility of attention, and that regime is small.

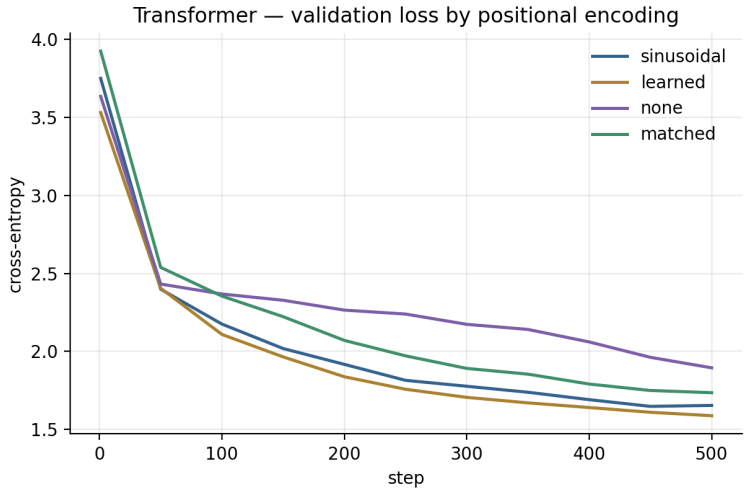


Figure 8: Positional ablation

Open threads. Warmup plus a cosine schedule, which would make the comparison above fair to the transformer — the obvious next run. Longer context, where the recurrent models must lose. RoPE instead of additive encodings. And KV caching, which is the difference between a sampler that is $O(T)$ per token and one that is $O(T^2)$.

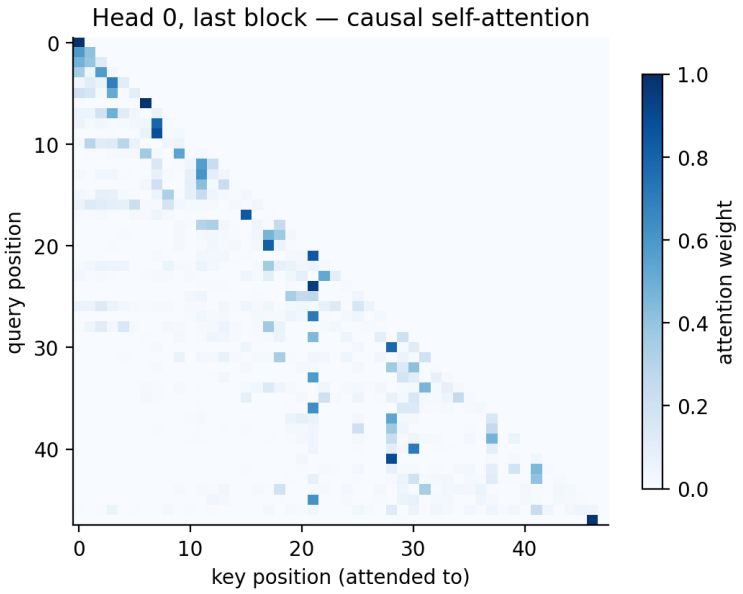


Figure 9: Attention map

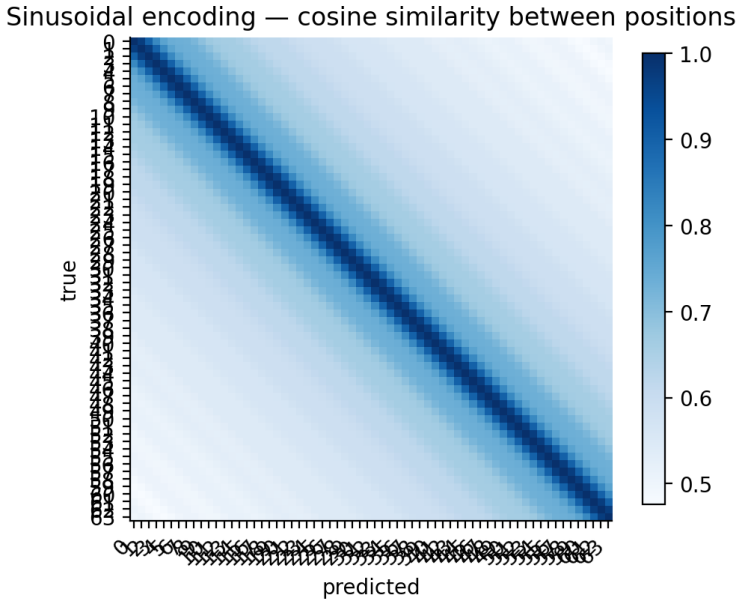


Figure 10: Positional similarity

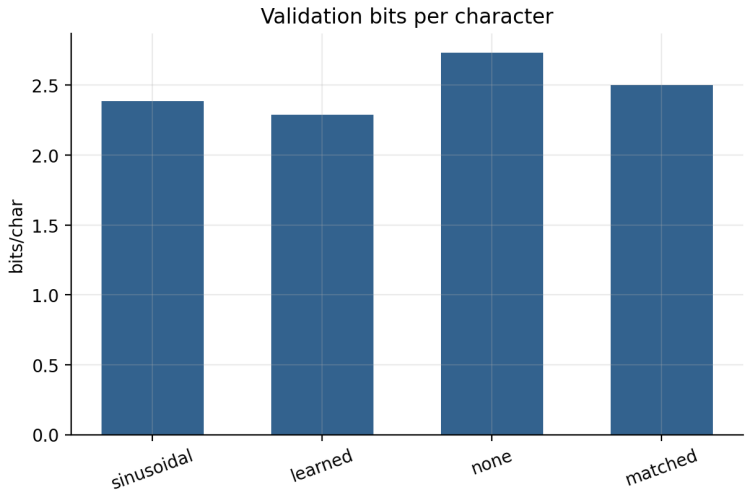


Figure 11: Bits per character

Reading: Attention Is All You Need; The Annotated Transformer; Xiong et al., *On Layer Normalization in the Transformer Architecture* (pre-LN vs post-LN).

C4 — Fine-tuning a small ViT

Question. A ViT is C3's block applied to image patches. Without a CNN's built-in priors, how much does it depend on pretraining?

The math

Patchification is the only image-specific step. An image $x \in \mathbb{R}^{H\times W\times C}$ is cut into $N = HW/P^2$ patches of side P, each flattened and projected:

$$z_0 = \left[x_{\text{cls}};\ x_p^1 E;\ x_p^2 E;\ \ldots;\ x_p^N E\right] + E_{\text{pos}}, \qquad E \in \mathbb{R}^{P^2 C\times d}.$$

After that it is C3's encoder verbatim. At 224^2 with $P = 16$ that is $N = 196$ tokens, and attention's $O(N^2)$ cost is why input resolution is expensive here in a way it is not for a CNN.

What the architecture does not assume. A convolution hard-codes locality (a unit sees a small neighbourhood) and translation equivariance ($f(\text{shift}(x)) = \text{shift}(f(x))$). Self-attention assumes neither: every patch can attend to every other from layer 1, and position enters only through E_{pos}, which is *learned*. That is more expressive and strictly less constrained — so with little data the ViT must learn from examples what a CNN gets for free, which is the standard explanation for ViTs being data-hungry and the thing this rung measures.

Probe versus fine-tune ask different questions. A linear probe on frozen features asks *are the pretrained representations already linearly separable for this task?* — it optimizes $\min_W \sum_i \ell(W\phi(x_i), y_i)$ with ϕ fixed, a convex problem in W. Fine-tuning asks *can the representation itself be improved?*, at 7000× the trainable parameters and the risk of destroying pretrained features with too large a step (hence the much smaller learning rate).

Results (ViT-Tiny, 4-class CIFAR subset — plane/car/ship/truck — 1500 train images, 2 epochs, medians over seeds 0–2):

arm	test accuracy	trainable parameters	seconds (T4)
from scratch	0.420	5.53M	12.8
linear probe	0.866	**772**	5.5
full fine-tune	**0.955**	5.53M	12.8

Pretraining is worth **+0.452** (probe over scratch, range 0.421–0.458); fine-tuning adds a further **+0.090** over the probe (range 0.073–0.100).

What surprised me. Not the ordering — the *ratio*. The linear probe trains **772 parameters**, four classes' worth of weights on a 192-dimensional feature, and lands within 9 points of a full fine-tune of 5.5 million. Essentially all of the task-relevant work was already done by ImageNet pretraining; the fine-tune is polishing. And the from-scratch arm at 0.420 is barely above the 0.25 chance of four classes — 1500 images is simply not enough to teach a transformer what an edge is.

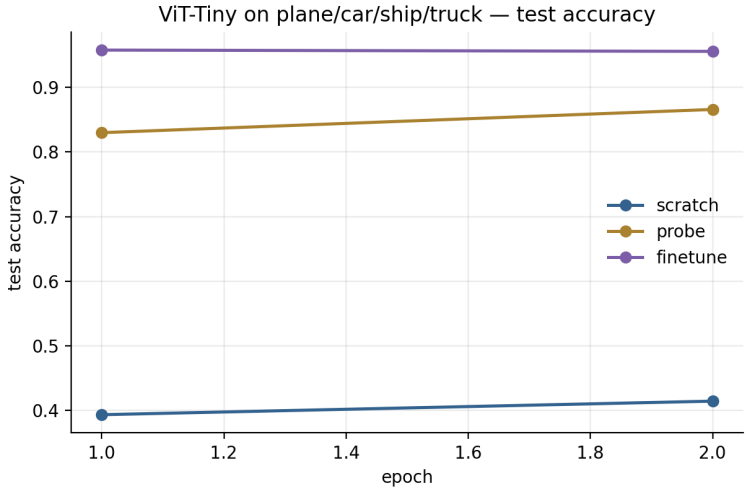


Figure 12: Arms

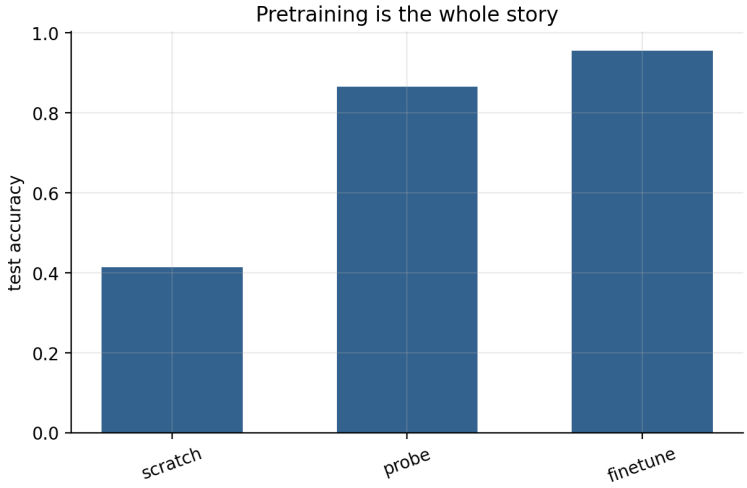


Figure 13: Final accuracy

Open threads. Freezing progressively more blocks, to find where "probe" turns into "fine-tune". Data augmentation, which is how the ViT paper's own from-scratch results were rescued. DINO or MAE features instead of supervised ImageNet ones — that comparison is part D's business.

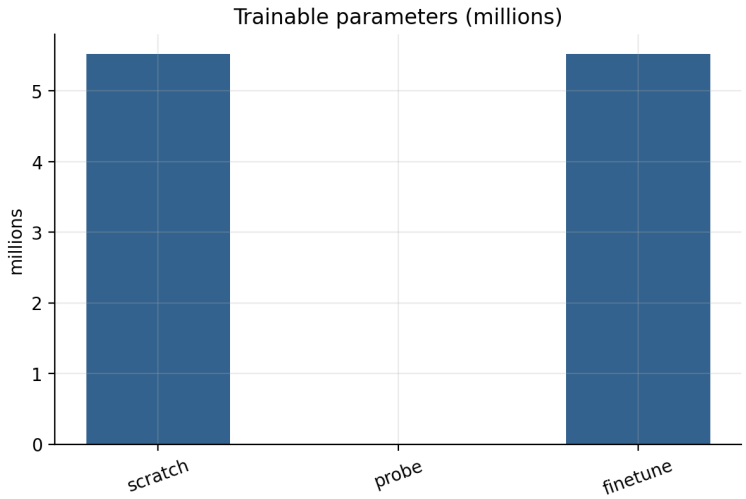


Figure 14: Trainable parameters

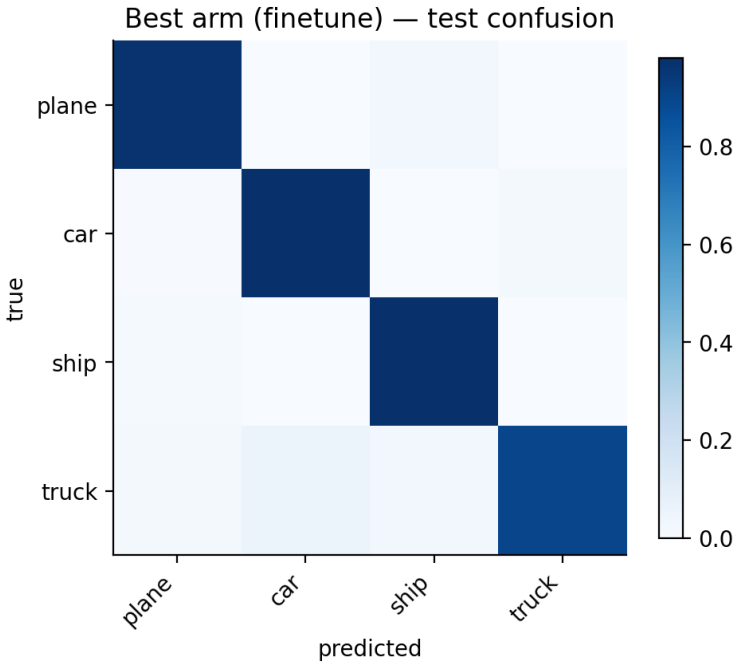


Figure 15: Confusion

Reading: An Image is Worth 16x16 Words (ViT); Steiner et al., *How to train your ViT*; timm.

C5 — What changed since C3, and what it cost

Question. C3's transformer is eight years old as a design. Every production decoder since has changed the attention layer. What are those changes actually buying, in a quantity that can be computed rather than argued about?

The math

The bill attention pays is not the quadratic term. At generation time a decoder emits one token at a time, and each new token attends over everything already produced — so every past key and value must be *retained* and re-read. The cache for a multi-head stack is

$$M_{\text{KV}} = 2\, b\, L\, H\, d_h\, T,$$

bytes for L layers, H heads of width d_h, context T and b bytes per element. It grows linearly with context and is read in full per token, which makes decode memory-bound: the arithmetic is trivial next to the bandwidth. Four families of change attack one factor each.

Grouped-query attention shrinks H to H_{kv} in that formula and nowhere else — queries keep their full head count, so the model's capacity to *ask* different questions is untouched while the memory of answers is shared.

Sliding-window attention replaces T with $\min(T, W)$, making a layer's cache constant past the window. Reach is not lost, it is relocated: stacking L such layers gives a receptive field of $L \cdot W$ through composition, the same trick a CNN uses for its effective field.

Multi-head latent attention stops caching per-head keys and values at all. DeepSeek-V2 projects them into one shared low-rank latent c_{KV} of rank r, plus a small decoupled rotary key that must stay outside the compression because RoPE does not commute with the down-projection:

$$M_{\text{MLA}} = b\, L\, (r + d_{\text{rope}})\, T.$$

The head count has left the expression entirely.

A delta rule leaves the cache paradigm. Gated DeltaNet keeps a fixed-size matrix $S \in \mathbb{R}^{H_v \times d_k \times d_v}$ updated by an outer-product rule, plus a short causal convolution window. Its size is chosen by the architect, not by how much text has been read — so the state is **constant in** T. That is C1's bargain exactly: compress the past into a fixed state. What changed is not the bargain but the budget, and the update rule that fills it.

What I built. No training. Five configurations — C3's own, measured from this repo, and four published ones recorded with their hub ids — reduced to layer lists and evaluated for decode state against context. Each mechanism is also ablated **against its own config with that mechanism removed**, rather than against a different model, so the number attributed to it is not really a difference in ten other things. The accounting lives in `src/mlladder/archspec.py` and is shared with H6.

Results (decode state in MiB, fp16):

decoder	layers	1k	16k	64k	256k
C3 (this repo)	4 full	3.0	48	192	768
Qwen3-8B	36 full (GQA 32:8)	144	2,304	9,216	36,864
Mistral-7B	32 sliding (W=4096)	128	**512**	**512**	**512**
DeepSeek-V2-Lite	27 MLA (r=512)	30	486	1,944	7,776
Qwen3-Next-80B-A3B	36 linear + 12 full	62	422	1,574	6,182

Each mechanism against its own config without it, at 64k tokens:

mechanism	cache shrink
GQA (32 query heads over 8 KV)	**4.0×**
sliding window 4096	**16.0×**
MLA (rank 512 + 64 rope)	**10.7×**
delta rule (vs its own all-attention twin)	**3.9×**

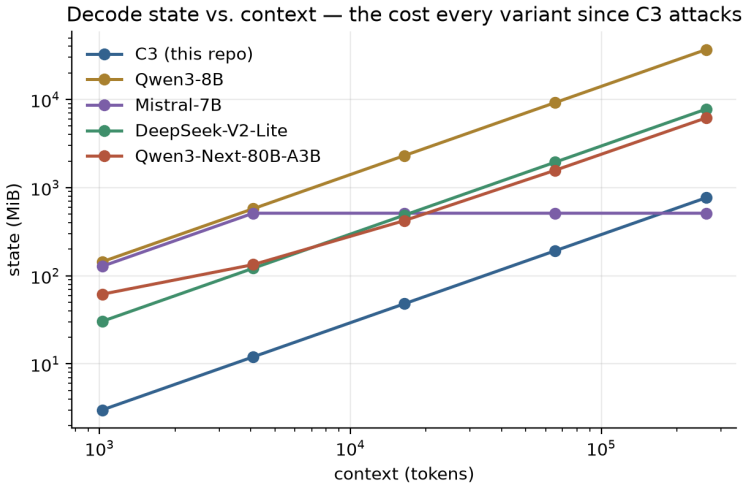


Figure 16: Decode state

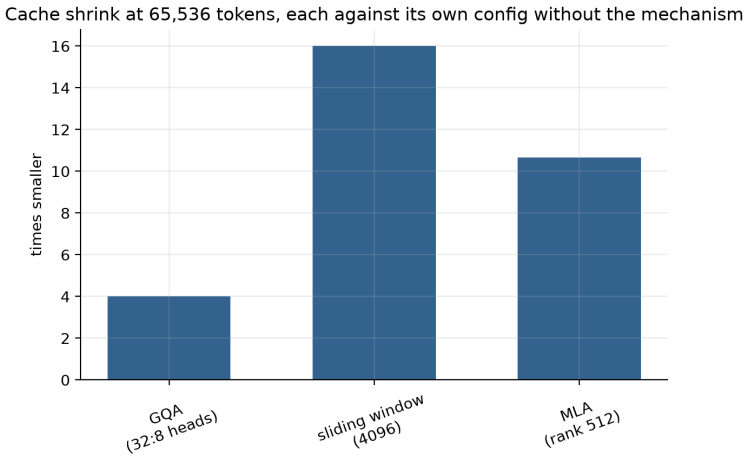


Figure 17: Mechanism savings

What surprised me. Three things.

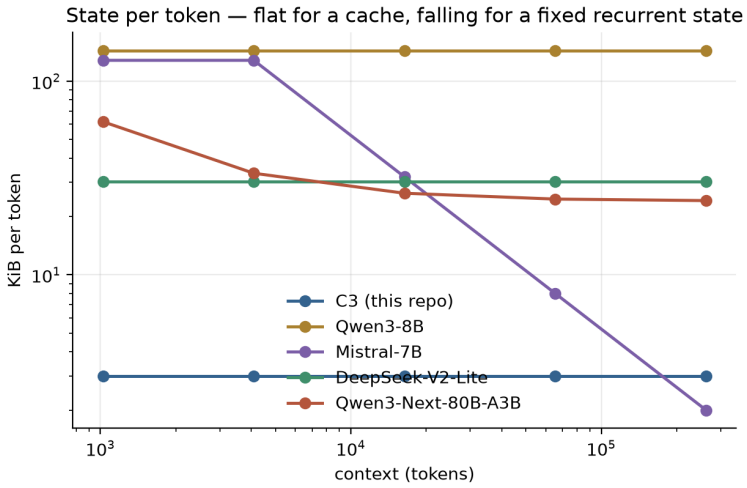


Figure 18: State per token

The numbers are enormous and they are not about parameters. Qwen3-8B carries **9.2 GiB of KV cache at 64k tokens** — larger than the 8B model's own weights in fp16. The thing that stops a decoder serving long contexts is usually not its size; it is the cache the context creates, and no amount of quantizing weights touches it.

Mistral's window is the bluntest instrument and the biggest win. A cache that simply stops growing beats every clever compression asymptotically: 512 MiB at 64k and still 512 MiB at 256k, while MLA's 10.7× compression is still growing linearly underneath it. And the reach is not obviously sacrificed — 32 layers at a 4096 window compose to a 131,072-token receptive field. The cost is that the reach is *indirect*, mediated through depth, which is exactly the compromise C1's recurrence made and lost on. Whether it loses here is not something state accounting can answer.

C1's answer came back, and the crossover is early. The delta-rule state is 1.05 MiB per linear layer, so Qwen3-Next carries **38 MiB before reading a single token** — strictly worse than any cache at short context. It pays that back against its own all-attention twin from **537 tokens** onward. I had expected a constant-state design to need a very long context to justify itself; at these dimensions it needs about two paragraphs. The tempting comparison — the hybrid against Qwen3-8B, which crosses at 322 tokens — is the one to avoid, because it credits the delta rule with savings that are really Qwen3-Next's much more aggressive GQA (2 KV heads against 8).

A check worth having. DeepSeek publishes V2-Lite at 15.7B total and 2.4B activated. The accounting here computes 15.8B (**0.6% off**) and, excluding the embedding and output head that DeepSeek's transformer-only figure omits, 2.32B (**3.2% off**). H6's independent check against TrOCR agreed to 1.3%. Two validations on models with known answers is what makes the other numbers in these tables worth reading.

Open threads. Gemma 4's ring-buffered sliding KV, which sizes the buffer at window + prefill chunk rather than the full context and is implemented in the accompanying fork — the natural next arm. Actually *measuring* decode throughput against these predictions, since memory-bound is a claim about bandwidth that a byte count only implies. And quality: none of this says whether a 512 MiB windowed cache answers questions as well as a 9 GiB full one, which is the question that decides anything.

Reading: GQA; Mistral 7B for sliding windows; DeepSeek-V2 for MLA; RoFormer; and candle's `qwen3_5.rs`, `deepseek2.rs` and `gemma4/` for the implementations.

What part C settled

1. **Every architecture here is a different answer to "how does information travel?"** Recurrence carries it in a state and pays multiplicatively per step; gating buys an additive path; attention refuses to compress and pays $O(T^2)$. The measurements line up with the algebra: the RNN's geometric Jacobian product shows up as a 27-step horizon, the LSTM's product of forget gates as 63, and attention's convex combination as a bottleneck-vs-attention gap that grows from 0.003 to 0.98 as the source gets longer.

2. **Reach is not the same as use.** The LSTM's gradient arrives 63 steps back and it still cannot solve 30-step recall at this budget, and its extra memory *loses* to a wide vanilla RNN on the actual language-modelling task. An architecture's capability and what a training run extracts from it are different quantities.

3. **The famous architecture lost, and the regime is why.** A 343k-parameter vanilla RNN beat a 1.8M-parameter transformer on 250k characters at 64 tokens of context with no warmup. Nothing about that contradicts the literature; it locates the boundary of it, which is more useful than a fourth reproduction of a result everyone already believes.

4. **Two metrics on the same run told opposite stories.** C2's bottleneck model keeps 71% character accuracy while getting 0.5% of dates right. Choosing the metric that matches the task's actual failure mode did more work here than any modelling decision.

5. **A seed does not pin a number, and which numbers it fails to pin is informative.** Across three full sweeps — one on CPU, two on separate T4 sessions — C1, C3 and C4 reproduced their medians *exactly* (bits/char, horizons, accuracies, the causal-leak zero). C2's bottleneck exact-match did not: 0.363, 0.349 and 0.326 at 101 characters, 0.012, 0.017 and 0.005 at 193. Seeding does not make cuDNN's backward deterministic, and the bottleneck arm sits exactly at the edge of failing, where an all-or-nothing metric over ten characters amplifies any tiny difference in weights. Its character accuracy, measuring the same runs, moved by only ~0.015. So the metric that made the failure legible is also the least stable one — both facts follow from it being a threshold measure near a threshold. The conclusion (the bottleneck collapses while attention does not) is unaffected on every run; the third decimal place is fiction. Figures and metrics here therefore come from one sweep on one device, and `--check` should be run on that device.

6. **The design C1 lost with came back, and the ladder is why I noticed.** C5's delta-rule stack is C1's bargain — compress the past into a fixed state — with a larger budget and a better update rule, and it pays for itself from 537 tokens of context. Reading Qwen3-Next's config as "a linear-attention hybrid" says nothing; reading it as "C1, with the compression problem solved differently" is only available because this part built C1 first.

Cost of the part: ~2,300 lines including tests, 27 test cases, and a full three-seed sweep in about 8 minutes on a T4 (C1 67s, C2 25s, C3 33s, C4 35s per seed; C5 is arithmetic) — the first part where the CPU-only path stopped being comfortable.

Figures regenerate with `uv run python tools/writeup_figures.py C`; the numbers they are quoted against live in `data/c-metrics.json`. The part README is the short version of this document.